

# Reinforcement Learning

## Mountain Car Continuous

**Nang Cherry Naw, Viktória Dobišová**

ACS 301  
2025/2026

## 1. Introduction

This project explores how Reinforcement Learning (RL) algorithms can be applied to solve the MountainCarContinuous-v0 environment from the Gymnasium library. The main goal is to assess whether custom reward offers advantages over the default reward system and to compare the performance of two algorithms: Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC).

## 2. Methodology

### a. Environment and Problem Statement

The MountainCarContinuous-v0 represents a classic continuous control challenge. The agent starts at the bottom of a valley (around position -0.5) and must figure out how to drive a car up a steep hill to reach a flag at position 0.45 or higher. Initially, the car does not have enough power to make it up the hill in one go. Instead, the agent needs to learn that moving backward first is the key to building enough momentum for the climb, which is a counterintuitive solution. The environment describes each state with two values: position (from -1.2 to 0.6) and velocity (from -0.07 to 0.07). The agent is also allowed to apply any force between -1 and 1 at each step. Rewards are sparse. Every move costs a small penalty based on the square of the action and only reaching the goal earns a bigger reward.

### b. Algorithms

PPO is one of the go-to algorithms in modern reinforcement learning, largely because it updates policies in a stable way. It keeps changes in check by clipping the difference between the new and old policy probabilities, which prevents wild swings during training. For the project, we used PPO for both the baseline and custom reward settings. Since PPO is an on-policy method, it learns from data collected while interacting with the environment.

To compare with PPO, we also tested SAC, which is an off-policy actor-critic algorithm. SAC learns from a replay buffer of past experiences rather than just the current data, making it more sample-efficient. This can be beneficial in continuous tasks such as MountainCarContinuous. Additionally, SAC has built-in entropy regularization that encourages exploration, which helps it maintain diverse action strategies during training.

For both algorithms, we set the entropy coefficient to 0.01 to limit excessive exploration, since our custom reward function already provides clear guidance to the agent. We trained each model for 500 000 timestamps. To keep track of progress, we saved checkpoints every 50 000 steps and evaluated performance using a maximum episode length of 1 500 steps.

### c. Custom Rewards

The default reward in MountainCarContinuous gives a penalty of -1 per step and a reward for reaching the goal, which does not give much guidance during training. To help the agent learn better, we designed a custom reward function with four components:

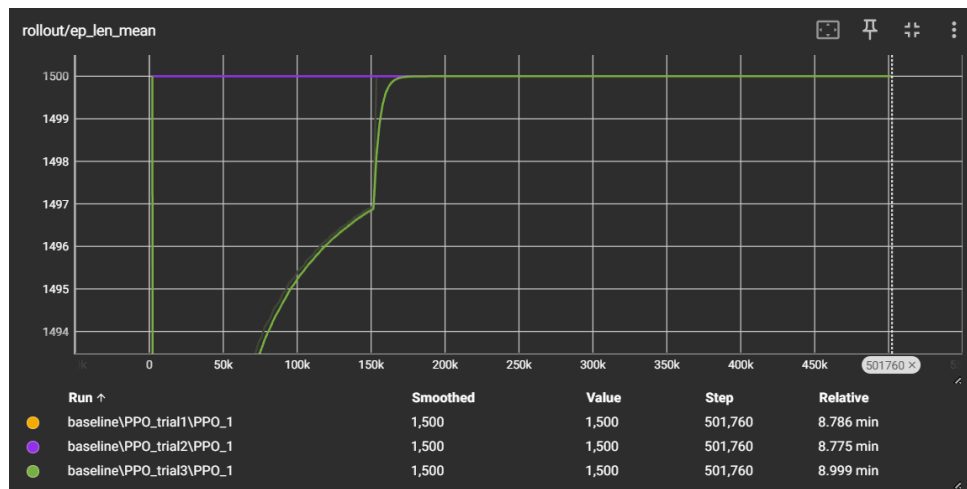
- **Distance to Goal Reward:** We give the agent a reward based on how close it gets to the target position. This provides feedback at each step, not just the end when it succeeds.
- **Velocity Bonus:** If the car is left of the goal and moving right, it gets a bonus. This encourages the agent to build momentum in the right direction.
- **Energy Penalty:** Large or unnecessary actions get punished, encouraging the agent to learn smoother control.
- **Success Bonus:** Reaching the goal still brings a large reward, keeping the agent focused on finishing the task.

### d. Extension - Algorithm Comparison

As our extension, we trained a SAC model with the same custom reward function and hyperparameters as PPO. This way, we could focus on the difference between the algorithms without other factors interfering. SAC reached the goal faster and used less energy per episode, showing that even with the same reward, the choice of algorithm affects learning efficiency and control smoothness. This comparison helped us see that reward shaping enables learning, but algorithm selection still matters for performance quality.

## 3. Results and Discussion

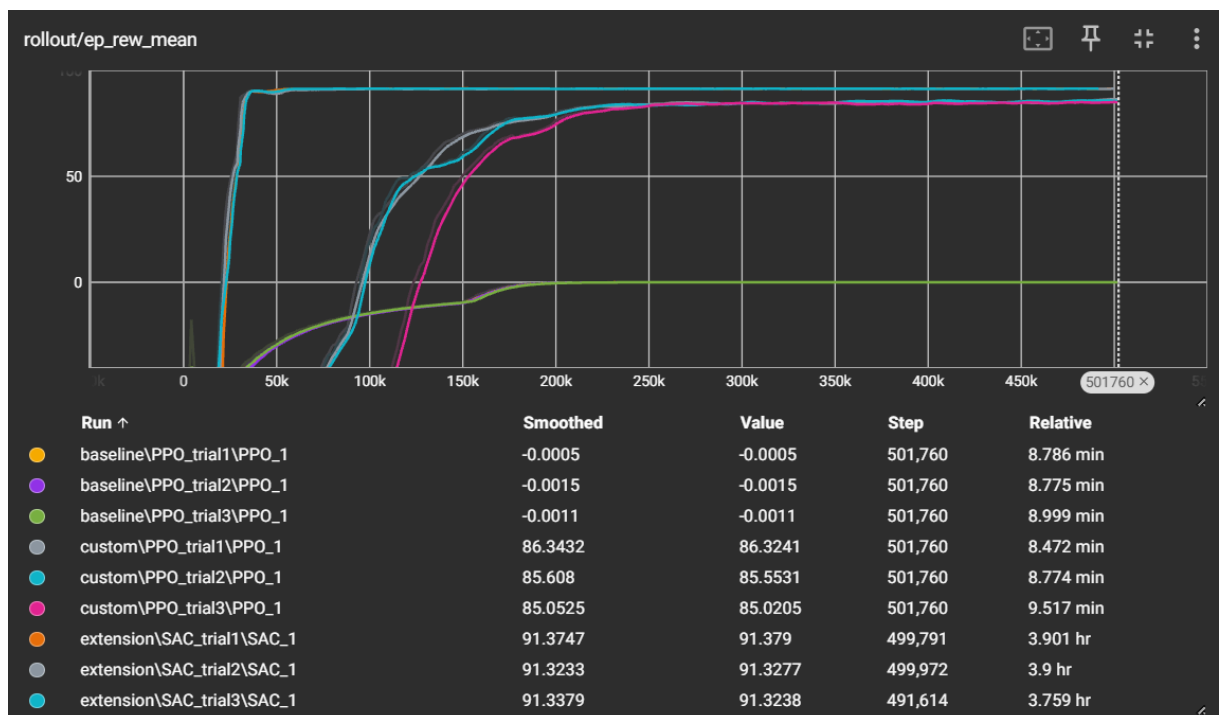
### a. Analysis of Default Reward Performance



The baseline agent trained with PPO using default rewards completely failed. Across all three trails, the agent achieved a mean reward of 0 with zero

successful episodes out of all evaluations. The agent never reached the goal position, only getting as close as -0.464 on average. Episodes always ran for the full 1500 steps (the maximum episode length), meaning the agent never discovered any useful behaviour. Looking at the training curves, PPO showed almost no improvement over the entire 500 000 timestamps, rewards stayed flat and episode lengths never decreased. These results show that the default reward structure doesn't provide enough guidance for the agent to learn in this environment.

## b. Training Curves and Performance Metrics



The training curves clearly show how much impact the reward design had on learning. The baseline curves are basically flat the entire time, no improvement, no learning happening. But with custom rewards, we see the difference right away. Both PPO and SAC's reward curves shoot up significantly in the first 100 000 steps, then level off. PPO stabilizes around 85 reward, SAC around 91. The episode length curves drop sharply too as the agents are figuring out faster and faster ways to reach the goal.

One thing that stands out is how quickly SAC's curves flatten out compared to PPO's. SAC reaches its best performance faster during training, then stays there. PPO takes longer to get to the same level. This might explain why SAC ended up with lower variance, it sticks to the good solution it found rather than still exploring different strategies late in training.

- c. Comparative Analysis of Algorithms
  - Default Reward vs Custom Reward

| BASELINE:                                          | CUSTOM:                                 |
|----------------------------------------------------|-----------------------------------------|
| Overall Mean Reward: -0.000                        | Overall Mean Reward: 85.524             |
| Overall Std Reward: 0.000                          | Overall Std Reward: 2.380               |
| Overall Mean Ep Len: 1500.000                      | Overall Mean Ep Len: 113.767            |
| Overall Mean Energy: 0.001                         | Overall Mean Energy: 81.073             |
| Overall Success Rate: 0.000                        | Overall Success Rate: 100.000           |
| Overall Reward Per Energy: -0.100                  | Overall Reward Per Energy: 1.118        |
| <u>C:\KDG\reinforcement_learning\src\evaluate.</u> | Overall Mean Steps To Success: 113.767  |
| print(f" Overall {key.replace('_', ' ')}.          | Overall Mean Energy Per Success: 81.073 |
| Overall Mean Steps To Success: nan                 | Overall Mean Action Magnitude: 0.790    |
| Overall Mean Energy Per Success: nan               | Overall Mean Action Smoothness: 0.038   |
| Overall Mean Action Magnitude: 0.001               | Overall Mean Max Position: 0.493        |
| Overall Mean Action Smoothness: 0.000              |                                         |
| Overall Mean Max Position: -0.464                  |                                         |

The default reward didn't work at all. With it, PPO got a mean reward of 0, never reaching the goal and always running the full 1500 steps. The custom reward completely changed everything: success rate went from 0% to 100%, mean reward jumped to 85.524 and the agent solved the problem in just 113.767 steps on average. The agent also actually started moving (action magnitude went from 0.001 to 0.79, showing it learned control instead of just waiting and doing nothing in hopes to mitigate penalties. This proves that a proper reward system is extremely important, without it the agent has no idea what it is supposed to do.

- Custom Reward With and Without the Extension

| CUSTOM:                                 | EXTENSION:                              |
|-----------------------------------------|-----------------------------------------|
| Overall Mean Reward: 85.524             | Overall Mean Reward: 91.629             |
| Overall Std Reward: 2.380               | Overall Std Reward: 0.320               |
| Overall Mean Ep Len: 113.767            | Overall Mean Ep Len: 69.633             |
| Overall Mean Energy: 81.073             | Overall Mean Energy: 49.459             |
| Overall Success Rate: 100.000           | Overall Success Rate: 100.000           |
| Overall Reward Per Energy: 1.118        | Overall Reward Per Energy: 1.860        |
| Overall Mean Steps To Success: 113.767  | Overall Mean Steps To Success: 69.633   |
| Overall Mean Energy Per Success: 81.073 | Overall Mean Energy Per Success: 49.459 |
| Overall Mean Action Magnitude: 0.790    | Overall Mean Action Magnitude: 0.815    |
| Overall Mean Action Smoothness: 0.038   | Overall Mean Action Smoothness: 0.042   |
| Overall Mean Max Position: 0.493        | Overall Mean Max Position: 0.463        |

Both PPO and SAC could solve the task with the custom reward and hit 100% success. However, SAC was better at it. SAC reached the goal in 69.633 steps on average while PPO took 113.767 steps (38.8% faster). SAC also used a lot less energy: 49.459 units versus PPO's 81.073 units (39% less).

The biggest difference though is consistency. SAC's results barely varied across trials (std: 0.32) but PPO's results were all over the place

(std: 2.38). This means SAC found a solid solution every time, while PPO's performance depends a lot on luck. Both algorithms pushed the car with similar force (action magnitude is around 0.8), so it is not like SAC just does bigger movements, it's smarter about how it explores and learns. This shows that regardless of a good reward function, picking the right algorithm still matters.

#### d. Challenges Faced and Solutions Implemented

We ran into several challenges during the project. Initially, we chose SAC as our baseline and custom algorithm but training took extremely long to complete. We swapped it for PPO because we had to run it several times until we got the reward system to decently perform.

For the extension, we first tried using A2C as an alternative since it is even faster but it performed very poorly compared to the custom PPO. We switched back to SAC for the extension, which gave us much better results but ate up a lot of time.

Training time was also an issue because we started using Google Colab for GPU but it kept running out of memory and crashing mid-training. The models were not saved properly either, so we could not resume training. We ended up running the code on the CPU in our local terminal instead, which was slower but much more reliable.

## 4. Conclusion

### a. Summary of Findings and Contributions

The results of this project show that custom reward shaping is essential for solving the MountainCarContinuous-v0 environment. With the default reward, the agent failed to learn anything useful, while the custom reward enabled both PPO and SAC to consistently reach the goal. The extension experiment showed that SAC outperformed PPO in terms of speed and energy efficiency, even though both algorithms were successful. Overall, the project highlights how important reward design and algorithm choice are in continuous control problems.

### b. Recommendations for Future Work

For future work, we could try tuning the entropy coefficient higher for PPO to see if it explores better and matches SAC more closely. We could also test other reward systems or try learning the reward function automatically instead of designing it by hand - this however is a bit more advanced.